## Exp4: Create a Docker image for a Node.js application that serves a static HTML page

### Software Requirements

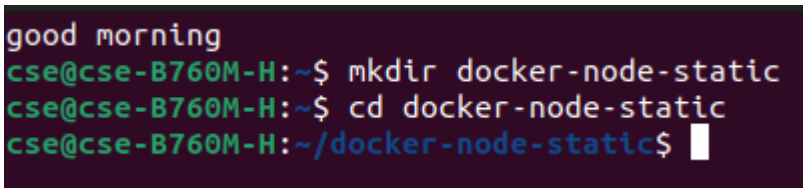
- Ubuntu 22.04 LTS
- Docker Engine
- Web browser

*(Node.js will be inside the container – no host installation required)*

### Step 1: Create Project Directory

```
mkdir docker-node-static
```

```
cd docker-node-static
```




### Step 2: Create Static HTML Page

Create **index.html**:

```
nano index.html
```

Paste:

```
<!DOCTYPE html>
<html>
<head>
    <title>Docker Node App</title>
</head>
<body>
    <h1>Hello from Node.js Docker Container!</h1>
    <p>This static page is served using Docker.</p>
</body>
</html>
```

```
  GNU nano 6.2
<!DOCTYPE html>
<html>
<head>
    <title>Docker Node App</title>
</head>
<body>
    <h1>Hello from Node.js Docker Container!</h1>
    <p>This static page is served using Docker.</p>
</body>
</html>
```

Save and exit.

**Step 3: Create Node.js Server File**

Create **server.js**:

```
nano server.js
```

Paste:

```
const http = require("http");
const fs = require("fs");
const path = require("path");

const port = 3000;

const server = http.createServer((req, res) => {
    const filePath = path.join(__dirname, "index.html");
    fs.readFile(filePath, (err, content) => {
        if (err) {
            res.writeHead(500);
            res.end("Error loading page");
        } else {
            res.writeHead(200, { "Content-Type": "text/html" });
            res.end(content);
        }
    });
});

server.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
});
```

```
  GNU nano 6.2
const http = require("http");
const fs = require("fs");
const path = require("path");

const port = 3000;

const server = http.createServer((req, res) => {
    const filePath = path.join(__dirname, "index.html");
    fs.readFile(filePath, (err, content) => {
        if (err) {
            res.writeHead(500);
            res.end("Error loading page");
        } else {
            res.writeHead(200, { "Content-Type": "text/html" });
            res.end(content);
        }
    });
});

server.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
});
```

Save and exit.

**Step 4: Create Dockerfile**

Create **Dockerfile**:

```
nano Dockerfile
```

Paste:

```
FROM node:18-alpine

WORKDIR /app

COPY index.html .
COPY server.js .

EXPOSE 3000

CMD ["node", "server.js"]
```

```
  GNU nano 6.2
FROM node:18-alpine

WORKDIR /app

COPY index.html .
COPY server.js .

EXPOSE 3000

CMD ["node", "server.js"]
```

Save and exit.

**Step 5: Build Docker Image**

```
docker build -t node-static-app .
```

```
cse@cse-B760M-H:~/docker-node-static$ docker build -t node-static-app .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.608kB
Step 1/6 : FROM node:18-alpine
18-alpine: Pulling from library/node
f18232174bc9: Pulling fs layer
dd71dde834b5: Pulling fs layer
1e5a4c89cee5: Pulling fs layer
25ff2da83641: Pulling fs layer
25ff2da83641: Waiting
1e5a4c89cee5: Verifying Checksum
1e5a4c89cee5: Download complete
f18232174bc9: Download complete
f18232174bc9: Pull complete
dd71dde834b5: Verifying Checksum
dd71dde834b5: Download complete
dd71dde834b5: Pull complete
1e5a4c89cee5: Pull complete
25ff2da83641: Verifying Checksum
25ff2da83641: Download complete
25ff2da83641: Pull complete
Digest: sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
Status: Downloaded newer image for node:18-alpine
 ---> ee77c6cd7c18
Step 2/6 : WORKDIR /app
 ---> Running in 2340645f80fb
 ---> Removed intermediate container 2340645f80fb
 ---> f81ee3526702
Step 3/6 : COPY index.html .
 ---> b3f695332f2e
Step 4/6 : COPY server.js .
 ---> e1acf66638ef
Step 5/6 : EXPOSE 3000
 ---> Running in 94dc8a7c15ff
 ---> Removed intermediate container 94dc8a7c15ff
 ---> 96b476143d57
Step 6/6 : CMD ["node", "server.js"]
 ---> Running in 9a909931ff93
 ---> Removed intermediate container 9a909931ff93
 ---> b485ad226199
Successfully built b485ad226199
Successfully tagged node-static-app:latest
cse@cse-B760M-H:~/docker-node-static$
```

docker images

```
cse@cse-B760M-H:~/docker-node-static$ docker images
REPOSITORY          TAG         IMAGE ID       CREATED              SIZE
node-static-app     latest      b485ad226199   About a minute ago   127MB
python-fileops      latest      fe055b321a21   23 hours ago         122MB
simple-web-app      latest      5557786ca37d   3 days ago           134MB
hello-world-image   latest      b7e187b4b9ab   3 days ago           77.9MB
python              3.10-slim   988b1bc80158   12 days ago          122MB
ubuntu              22.04       65c77cbc27c2   5 weeks ago          77.9MB
node                18-alpine   ee77c6cd7c18   10 months ago        127MB
cse@cse-B760M-H:~/docker-node-static$
```

**Step 6: Run the Docker Container**

```
docker run -d -p 3000:3000 node-static-app
```

Explanation:

- `-d` → detached mode
- `-p 3000:3000` → port mapping

```
cse@cse-B760M-H:~/docker-node-static$ docker run -d -p 3000:3000 node-static-app
88d1a29c966543aae6e7f436de498e215e944e61f94c3e9f08ae50cbc37ca0f9
```

**Step 7: Verify Running Container**

```
docker ps
```

```
cse@cse-B760M-H:~/docker-node-static$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED         STATUS         PORTS                                       NAMES
88d1a29c9665   node-static-app   "docker-entrypoint.s…"   9 seconds ago   Up 9 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp   pensive_blackburn
cse@cse-B760M-H:~/docker-node-static$
```

**Step 8: Access the Web Application**

Open browser and visit:

```
http://localhost:3000
```

Expected Output:

- Web page showing
  **"Hello from Node.js Docker Container!"**

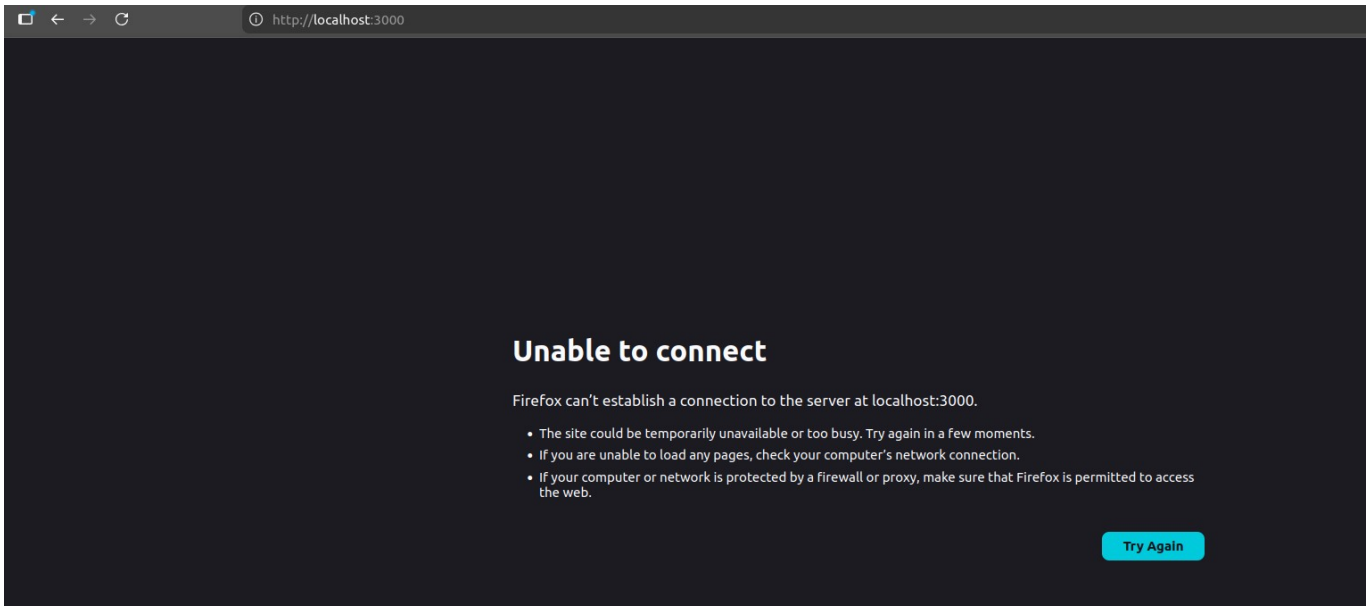
  **This static page is served using Docker.**

**Step 9: Stop the Container**

```
docker stop <container_id>
```

```
cse@cse-B760M-H:~/docker-node-static$ docker stop 88d1a29c9665
88d1a29c9665
cse@cse-B760M-H:~/docker-node-static$
```

Now go back to browser and confirm that the application is not running.




**Result**

A Docker image was successfully created and used to serve a static HTML page using a Node.js application.

***Note:***

***Why container keeps running?***

- *Node.js server runs continuously*
- *Foreground process stays alive*

***Why EXPOSE?***

- *Documents container port*
- *Helps in orchestration tools*

***Why port mapping?***

- *Container network is isolated*
- `-p` *exposes service to host*